PYTHON PROGRAMMING
C0_3 ASSINGMENT

NAME: SHABANA THASLEEM A.W

COURSE: PYTHON PROGRAMMING

COURSE CODE: CSA0802

ASSIGNMENT: 3

NATURE OF THE ASSIGNMENT: Individual

TOTAL MARKS: 50

SUBMISSION DATE: 12/11/26

PYTHON PROGRAMMING
C0_3 ASSINGMENT

Q1 – Lists and Slicing: Daily Rainfall Records (6 Marks)

Q1. COMPLETE PYTHON PROGRAM

```python
# Daily Rainfall Records

import random

# Generate rainfall readings for 365 days
readings = [random.randint(0, 100) for i in range(365)]

# Calculate weekly totals
weekly_totals = []

for i in range(0, len(readings), 7):
    week = readings[i:i+7]
    weekly_totals.append(sum(week))

print("Weekly Totals:", weekly_totals)

# Find wettest and driest weeks
wettest = max(weekly_totals)
driest = min(weekly_totals)

print("Wettest Week:", weekly_totals.index(wettest) + 1)
print("Maximum Rainfall:", wettest, "mm")

print("Driest Week:", weekly_totals.index(driest) + 1)
print("Minimum Rainfall:", driest, "mm")
```

```
Weekly Totals: [463, 437, 265, 193, 320, 270, 233, 357, 310, 160, 326, 237, 359,
279, 330, 322, 256, 387, 388, 528, 272, 351, 320, 329, 298, 407, 406, 307, 361,
313, 447, 269, 404, 301, 379, 298, 426, 399, 541, 392, 370, 332, 376, 342, 395,
233, 408, 307, 406, 498, 371, 316, 92]
Wettest Week: 39
Maximum Rainfall: 541 mm
Driest Week: 53
Minimum Rainfall: 92 mm
7-day Moving Average: [66.14285714285714, 61.57142857142857, 63.0, 59.0,
55.142857142857146, 53.285714285714285, 52.714285714285715, 62.42857142857143,
61.42857142857143, 59.714285714285715]
Dry Days: [25, 65, 69, 126, 147, 209]
```

2. SAMPLE INPUT AND OUTPUT

```python
readings = [
    10, 20, 30, 40, 50, 60, 70,
    0, 0, 10, 10, 10, 10, 10,
    5, 5, 5, 5, 5, 5, 5
]
```

3.PSEUDOCODE FOR 7-DAY MOVING AVERAGE

PYTHON PROGRAMMING
C0_3 ASSINGMENT

```
START

INPUT rainfall readings

FOR i = 0 TO length(readings) - 7

    total = 0

    FOR j = i TO i + 6
        total = total + readings[j]
    END FOR

    average = total / 7

    PRINT average

END FOR

STOP
```

4.SLICING OPERATOR

```
readings = [10, 20, 30, 40, 50, 60, 70,
            80, 90, 100, 110, 120, 130, 140]
```

5.LIST OPERATION COSTS

Operation	Time complexiy
Indexing	O(1)
Slicing	O(k)
Append()	O(1) amortized
Insert()	O(n)
pop() at end	O(1)

PYTHON PROGRAMMING
C0_3 ASSINGMENT

6.NUMBERS OF ADDITIONS IN MOVING AVERAGE :

For n readings and a window size of 7:

Naive method:

Each window requires 7 additions.

Number of windows:

$$ n-6 $$

Total additions:

$$ 7(n-6) $$

Running-sum method:

- First window requires 6 additions.
- Every next window requires 2 additions: one addition and one subtraction.

Total arithmetic updates:

$$ 6+2(n-7)=2n-8 $$

If the initial total is counted as 7 additions instead:

$$ 7+2(n-7)=2n-7 $$

Both counting conventions show linear growth.

7. OPTIMIZED MOVING AVERAGE USING RUNNING SUM

PYTHON PROGRAMMING
C0_3 ASSINGMENT

```python
def moving_average(readings):

    if len(readings) < 7:
        return []

    result = []

    current_sum = sum(readings[:7])

    result.append(current_sum / 7)

    for i in range(7, len(readings)):

        current_sum += readings[i]
        current_sum -= readings[i-7]

        result.append(current_sum / 7)

    return result


readings = [10, 20, 30, 40, 50, 60, 70,
            80, 90, 100, 110, 120, 130, 140]

print("Moving Average:", moving_average(readings))
```

```
Moving Average: [40.0, 50.0, 60.0, 70.0, 80.0, 90.0, 100.0, 110.0]
```

8.EXPERIMENTAL INVESTIGATION

```python
import random
import time


def naive_average(data):

    result = []

    for i in range(len(data) - 6):
        result.append(sum(data[i:i+7]) / 7)

    return result


def running_average(data):

    if len(data) < 7:
        return []

    result = []

    total = sum(data[:7])
    result.append(total / 7)

    for i in range(7, len(data)):
```

```
Readings: 365
Naive time: 0.00014
Running sum time: 5e-05

Readings: 36500
Naive time: 0.02284
Running sum time: 0.01562

Readings: 365000
Naive time: 0.24723
Running sum time: 0.13945
```

PYTHON PROGRAMMING
C0_3 ASSINGMENT

9. TEST EMPTY LIST AND FEWER THAN 7 READINGS:

```python
def average(data):

    if len(data) < 7:
        return []

    result = []

    for i in range(len(data) - 6):
        result.append(sum(data[i:i+7]) / 7)

    return result


print(average([]))
print(average([10, 20, 30, 40, 50]))
```

```
[]
[]
```

10. ALTERNATIVE CONCEPT-USING DEQUE:

```python
from collections import deque

def deque_average(data):

    window = deque(maxlen=7)
    result = []

    for value in data:

        window.append(value)

        if len(window) == 7:
            result.append(sum(window) / 7)

    return result


data = [10, 20, 30, 40, 50, 60, 70, 80]

print(deque_average(data))
```

```
[40.0, 50.0]
```

PYTHON PROGRAMMING
C0_3 ASSINGMENT

11. DESIGN QUESTION-LATE RAINFALL READING:

```
readings = [10, 20, 30, 40, 50]

readings.insert(2, 25)

print(readings)
```

```
[10, 20, 25, 30, 40, 50]
```

12.ALTERNATIVE COMPLEXITY ANALYSIS:

- `collections.deque(maxlen=7)` is useful for maintaining a 7-day window.
- A running sum with deque allows O(1) updates per new reading.
- A dictionary keyed by date provides average O(1) lookup by date.
- A dictionary does not automatically maintain chronological order for calculations; dates must be processed in order.

PYTHON PROGRAMMING
C0_3 ASSINGMENT

Q2 – Tuples, Mutability and Copying (7 Marks)

1. PYTHON PROGRAM-REPRESENTS NUS STOPS USING TUPLES:

```python
# Bus Stop Map using Tuples

# Store bus stops as tuples
stops = [
    (13.0827, 80.2707),
    (13.0604, 80.2496),
    (13.0475, 80.2090),
    (13.0878, 80.2785)
]

# Dictionary using tuples as keys
stop_names = {
    (13.0827, 80.2707): "Chennai Central",
    (13.0604, 80.2496): "Egmore",
    (13.0475, 80.2090): "Guindy",
    (13.0878, 80.2785): "Park Town"
}

print("Bus Stop Details:")

for latitude, longitude in stops:
    print(latitude, longitude,
          stop_names[(latitude, longitude)])
```

```
Bus Stop Details:
13.0827 80.2707 Chennai Central
13.0604 80.2496 Egmore
13.0475 80.209 Guindy
13.0878 80.2785 Park Town
```

2. CREATE A ZONE GRID CORRECTLY:

```python
rows = 3
cols = 3

grid = [[0] * cols] * rows

grid[0][0] = 5

print(grid)
```

```
[[5, 0, 0], [5, 0, 0], [5, 0, 0]]
```

3.DEMONSTATION OF SHALLOW COPY AND DEEP COPY :

PYTHON PROGRAMMING
C0_3 ASSINGMENT

```python
import copy

original = [[10, 20], [30, 40]]

# Shallow copy
shallow = copy.copy(original)

# Deep copy
deep = copy.deepcopy(original)

# Modify original
original[0][0] = 100

print("Original:", original)
print("Shallow Copy:", shallow)
print("Deep Copy:", deep)
```

```
Original: [[100, 20], [30, 40]]
Shallow Copy: [[100, 20], [30, 40]]
Deep Copy: [[10, 20], [30, 40]]
```

4. DEMONSTATION OF TUPLE UNPACKING:

```python
# Tuple unpacking

bus_stop = (13.0827, 80.2707)

latitude, longitude = bus_stop

print("Latitude:", latitude)
print("Longitude:", longitude)
```

```
Latitude: 13.0827
Longitude: 80.2707
```

5. WHY CAN TUPLES BE DICTIONARY KEYS BUT LISTS CANNOT?

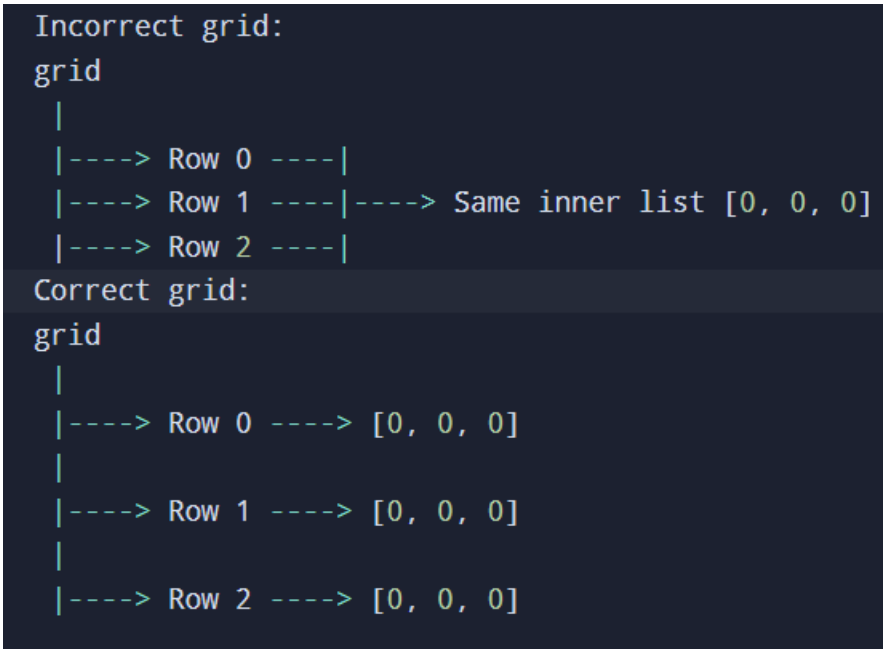
```python
# Tuple as dictionary key

location = {
    (13.0827, 80.2707): "Chennai"
}

print(location[(13.0827, 80.2707)])
```

```
Chennai
```

6. MEMORY DIAGRAM OF INCORRECT AND CORRECT GRIDS:



7. WHEN IS SHALLOW COPY ENOUGH AND WHEN IS DEEP COPY NEEDED?

Shallow copy:

Used when the outer collection needs to be copied but sharing nested objects is acceptable.

Deep copy:

Used when both outer and nested objects must be independent.

8.FUNCTION CHANGING A LIST ARGUEMENT:

PYTHON PROGRAMMING
C0_3 ASSINGMENT

```python
def update_list(data):
    data.append(100)


numbers = [10, 20, 30]

update_list(numbers)

print(numbers)
```

```
[10, 20, 30, 100]
```

9. DEMONSTRATE THE BUG USING is AND id ()

```python
# List aliasing demonstration

a = [[0] * 3] * 3

print("Same row:", a[0] is a[1])

print("ID of row 1:", id(a[0]))
print("ID of row 2:", id(a[1]))

# Correct grid
b = [[0 for j in range(3)] for i in range(3)]

print("Correct grid same row:", b[0] is b[1])
```

```
Same row: True
ID of row 1: 138608645214272
ID of row 2: 138608645214272
Correct grid same row: False
```

10.COMPARE MEMORY OF LIST AND TUPLE WITH 1,000,000 VALUES

PYTHON PROGRAMMING
C0_3 ASSINGMENT

```python
import sys

# Create list and tuple
my_list = list(range(1000000))
my_tuple = tuple(range(1000000))

print("List memory:",
      sys.getsizeof(my_list), "bytes")

print("Tuple memory:",
      sys.getsizeof(my_tuple), "bytes")
```

11. COMPARE CREATION TIME AND INDEXING TIME

```python
# Tuple creation
start = time.perf_counter()
my_tuple = tuple(range(1000000))
tuple_time = time.perf_counter() - start

# List indexing
start = time.perf_counter()

for i in range(10000):
    x = my_list[i]

list_index_time = time.perf_counter() - start

# Tuple indexing
start = time.perf_counter()

for i in range(10000):
    x = my_tuple[i]

tuple_index_time = time.perf_counter() - start

print("List creation time:", list_time)
print("Tuple creation time:", tuple_time)

print("List indexing time:", list_index_time)
print("Tuple indexing time:", tuple_index_time)
```

```
List creation time: 0.026897784002358094
Tuple creation time: 0.027215163019718602
List indexing time: 0.0009331690089311451
Tuple indexing time: 0.0008192590030375868
```

PYTHON PROGRAMMING
C0_3 ASSINGMENT

12. TEST CHANGING A LIST STORED INSIDE A TUPLE

```python
# List inside a tuple

data = ([10, 20, 30], 100)

print("Before:", data)

data[0].append(40)

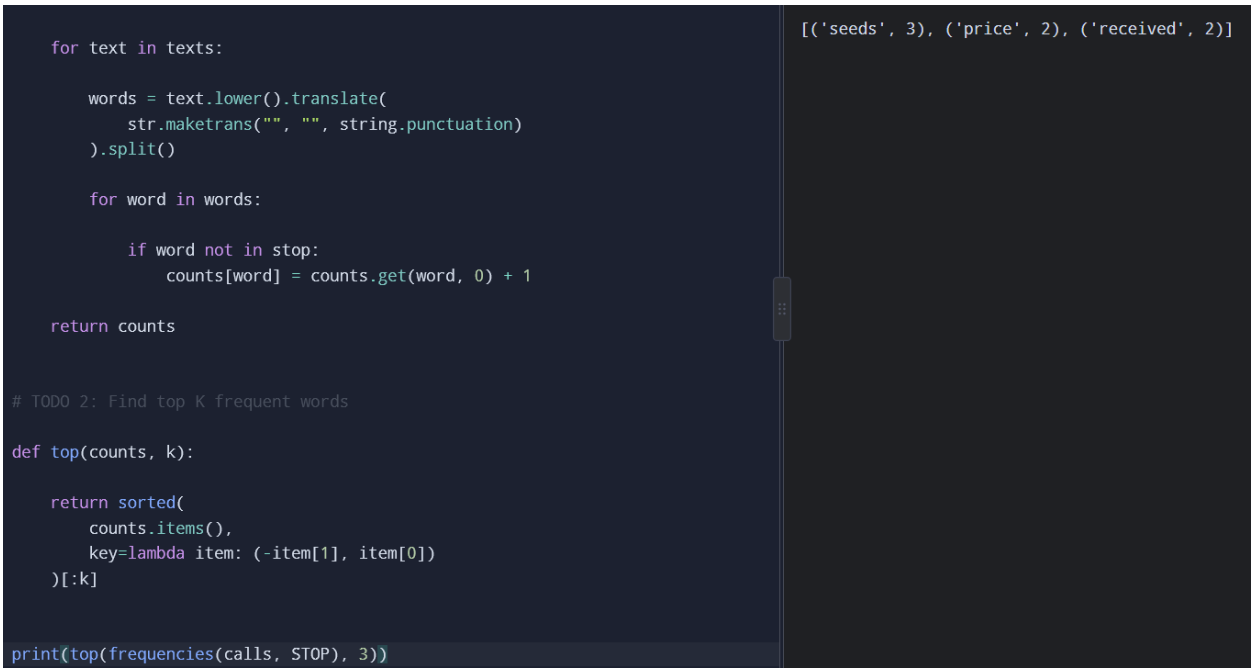
print("After:", data)
```

```
Before: ([10, 20, 30], 100)
After: ([10, 20, 30, 40], 100)
```

PYTHON PROGRAMMING
C0_3 ASSINGMENT

Q3 – Dictionaries: Farmer Helpline Complaint Analysis (7 Marks)

1.COMPARE PYTHON PROGRAMMING



```
    for text in texts:

        words = text.lower().translate(
            str.maketrans("", "", string.punctuation)
        ).split()

        for word in words:

            if word not in stop:
                counts[word] = counts.get(word, 0) + 1

    return counts


# TODO 2: Find top K frequent words

def top(counts, k):

    return sorted(
        counts.items(),
        key=lambda item: (-item[1], item[0])
    )[:k]


print(top(frequencies(calls, STOP), 3))
```

```
[('seeds', 3), ('price', 2), ('received', 2)]
```

2.INPUT

```
seeds not received seeds delayed
fertiliser price high
seeds price high subsidy not received
```

3.OUTPUT

```
[('seeds', 3), ('price', 2), ('received', 2)]
```

Explanation:

- `seeds` occurs 3 times.
- `price` occurs 2 times.
- `received` occurs 2 times.
- `not` and `high` are removed because they are stop words.

Why does price come before received?

Both words have the same frequency of 2. When frequencies are equal, words are sorted alphabetically. Therefore, `price` comes before `received`.

4.GROUP COMPLAINTS BY DISTRICT AND CROP

```python
from collections import defaultdict

complaints = [
    ("Chennai", "Rice", "Seeds not received"),
    ("Madurai", "Wheat", "Fertiliser price high"),
    ("Chennai", "Rice", "Seeds delayed"),
    ("Salem", "Cotton", "Water supply problem"),
    ("Madurai", "Wheat", "Subsidy not received")
]

districts = defaultdict(list)
crops = defaultdict(list)

for district, crop, complaint in complaints:

    districts[district].append(complaint)
    crops[crop].append(complaint)

print("District-wise complaints:")
for district, data in districts.items():
    print(district, ":", data)

print("\nCrop-wise complaints:")
for crop, data in crops.items():
    print(crop, ":", data)
```

```
District-wise complaints:
Chennai : ['Seeds not received', 'Seeds delayed']
Madurai : ['Fertiliser price high', 'Subsidy not received']
Salem : ['Water supply problem']

Crop-wise complaints:
Rice : ['Seeds not received', 'Seeds delayed']
Wheat : ['Fertiliser price high', 'Subsidy not received']
Cotton : ['Water supply problem']
```

5.DICTIONARY LOOKUP EXPLANATION

A dictionary uses a hash table to locate keys.

- A hash function converts a key into a hash value.
- The hash value helps locate the corresponding entry.
- Average lookup time is O(1).
- In an unusual collision-heavy case, lookup can take O(n).

PYTHON PROGRAMMING
C0_3 ASSINGMENT

The average lookup cost does not depend directly on the dictionary size because hashing provides direct access to the relevant location.

6.DICTIONARY COUNTING VERSUS LIST.CONT ()

Method	Time complexity
Dictionary counting	O(n) average
list.count() for every distinct word	O(n × d)
collections.Counter	O(n) average

Here:

- n = total number of words.
- d = number of distinct words.

Dictionary counting is faster because it processes each word once on average.

7.DICTIONARY KEY, VALUE AND ITEM VIEWS

```
data = {
    "seeds": 3,
    "price": 2,
    "received": 2
}

print(data.keys())
print(data.values())
print(data.items())
```

```
dict_keys(['seeds', 'price', 'received'])
dict_values([3, 2, 2])
dict_items([('seeds', 3), ('price', 2), ('received', 2)])
```

PYTHON PROGRAMMING
C0_3 ASSINGMENT

8.EXPEIMENTAL INVESTIGATION

```
    return result


for n in [1000, 10000, 100000]:

    data = [random.choice(words) for i in range(n)]

    start = time.perf_counter()
    a = dictionary_count(data)
    t1 = time.perf_counter() - start

    start = time.perf_counter()
    b = list_count(data)
    t2 = time.perf_counter() - start

    start = time.perf_counter()
    c = dict(Counter(data))
    t3 = time.perf_counter() - start

    print("\nComplaints:", n)
    print("Dictionary:", round(t1, 6))
    print("list.count():", round(t2, 6))
    print("Counter:", round(t3, 6))

    print("Results equal:", a == b == c)
```

```
Complaints: 1000
Dictionary: 6.3e-05
list.count(): 0.000104
Counter: 0.000205
Results equal: True

Complaints: 10000
Dictionary: 0.000748
list.count(): 0.000961
Counter: 0.000497
Results equal: True

Complaints: 100000
Dictionary: 0.007619
list.count(): 0.009625
Counter: 0.005076
Results equal: True
```

9.TEST EMPTY TEXT, UPPERCASE AND PUNCTUATION

```
calls = [
    "",
    "SEEDS, seeds!",
    "Price HIGH. price?"
]

STOP = {"high"}

print(frequencies(calls, STOP))
```

10.ENGINEERING DECISIONS

PYTHON PROGRAMMING
C0_3 ASSINGMENT

For 100,000 complaints per day, I would choose dictionary counting or `collections.Counter`.

Reasons:

1. Average O(n) time complexity.
2. Avoids repeated scanning.
3. Uses efficient key lookup.
4. Suitable for large complaint datasets.

Conclusion: Dictionary counting is efficient for complaint analysis because it reduces repeated searching and provides fast word-frequency calculation.

PYTHON PROGRAMMING
C0_3 ASSINGMENT

Q4 – Sets: Elective Registration Clash Detection (7 Marks)

1.COMPLETE PYTHON PROGRAM USING SET OPERATIONS

```python
iot_elective = {
    "192311002",
    "192311004",
    "192311009",
    "192311010"
}

all_students = {
    f"1923110{i:02d}"
    for i in range(1, 11)
}

# TODO 1: Set operations

clash = ai_elective & iot_elective

exactly_one = ai_elective ^ iot_elective

neither = all_students - (ai_elective | iot_elective)

print("clash:", sorted(clash))

print("exactly one:", sorted(exactly_one))

print("neither:", sorted(neither))
```

```
clash: ['192311004', '192311009']
exactly one: ['192311001', '192311002', '192311007', '192311010']
neither: ['192311003', '192311005', '192311006', '192311008']
```

2.INPUT

```
AI students:
192311001
192311004
192311007
192311009

IOT students:
192311002
192311004
192311009
192311010
```

3.OUTPUT

PYTHON PROGRAMMING
C0_3 ASSINGMENT



4.SET OPERATIONS EXPLANATIONS

Operator

Meaning

Python

Union

Students in either elective

`A \| B`

Intersection

Students in both electives

`A & B`

Difference

Students only in A

`A - B`

Symmetric difference

PYTHON PROGRAMMING
C0_3 ASSINGMENT

Students in exactly one

5.VENN DIAGRAM

- Intersection represents students registered in both electives.
- Symmetric difference represents students registered in exactly one elective.
- Neither represents students outside both circles but inside the universal set.

6.REWRITE USING SET METHODS

```
clash = ai_elective.intersection(iot_elective)

exactly_one = ai_elective.symmetric_difference(iot_elective)

neither = all_students.difference(
    ai_elective.union(iot_elective)
)

print("clash:", sorted(clash))
print("exactly one:", sorted(exactly_one))
print("neither:", sorted(neither))
```

7.WHY DO SETS AUTOMATTICALLY REMOVE DUPLICATES?

```
numbers = {10, 20, 20, 30, 30}

print(numbers)
```

```
{10, 20, 30}
```

8.MEMBERSHIP TESTING

```
students_list = ["101", "102", "103", "104"]

students_set = {"101", "102", "103", "104"}

print("102" in students_list)
print("102" in students_set)
```

```
True
True
```

PYTHON PROGRAMMING
C0_3 ASSINGMENT

9.WHY CAN SETS CONTAIN TUPLES BUT NOT LISTS?

```
valid = {(10, 20), (30, 40)}

print(valid)
```

```
{(30, 40), (10, 20)}
```

10.EXPERIMENTAL INVESTIGATION

```
import time

for n in [10000, 100000, 1000000]:
    data_list = list(range(n))
    data_set = set(range(n))

    target = n - 1

    start = time.perf_counter()

    for i in range(1000):
        result = target in data_list

    list_time = time.perf_counter() - start

    start = time.perf_counter()

    for i in range(1000):
        result = target in data_set

    set_time = time.perf_counter() - start

    print("\nStudents:", n)
    print("List search:", list_time)
    print("Set search:", set_time)
```

```
Students: 10000
List search: 0.10066098906099796
Set search: 9.712018072605133e-05

Students: 100000
List search: 1.1270169629715383
Set search: 0.00012400001287460327

Students: 1000000
List search: 10.488973040133715
Set search: 0.00010459986515343189
```

11.COMPARE NESTED LOOPS AND SET INTERSECTION

PYTHON PROGRAMMING
C0_3 ASSINGMENT

```python
def nested_common(a, b):

    result = []

    for x in a:
        for y in b:
            if x == y:
                result.append(x)

    return result


def set_common(a, b):

    return a & b


a = list(range(10000))
b = list(range(5000, 15000))

print("Nested loop:", len(nested_common(a, b)))
print("Set method:", len(set_common(set(a), set(b))))
```

```
Nested loop: 5000
Set method: 5000
```

PYTHON PROGRAMMING
C0_3 ASSINGMENT

Q5 – List Comprehension: Cleaning Sensor Readings (6 Marks)

1. COMPLETE PYTHON PROGRAM USING A FOR LOOP

```
a# Temperature Sensor Monitoring

readings = [4, 5, 55, 1, -60, 8, 9, 3, 50, -50, 2]

valid = []
fahrenheit = []
alarms = []

for minute, temp in enumerate(readings, start=1):

    if -50 <= temp <= 50:

        valid.append(temp)

        f = (temp * 9/5) + 32
        fahrenheit.append(f)

        if temp < 2 or temp > 8:
            alarms.append(minute)

print("Valid Celsius:", valid)
print("Fahrenheit:", fahrenheit)
print("Alarm Minutes:", alarms)
```

2.INPUT

```
[4, 5, 55, 1, -60, 8, 9, 3, 50, -50, 2]
```

PYTHON PROGRAMMING
C0_3 ASSINGMENT

3.OUTPUT

```
Valid Celsius:
[4, 5, 1, 8, 9, 3, 50, -50, 2]

Fahrenheit:
[39.2, 41.0, 33.8, 46.4,
 48.2, 37.4, 122.0, -58.0, 35.6]

Alarm Minutes:
[4, 7, 8, 9, 10]
```

Explanation:

- 55 and -60 are invalid readings.
- -50 and 50 are valid because the boundary values are included.
- Temperatures below 2°C or above 8°C generate alarms.

4.USING LIST COMPREHENSION

```
readings = [4, 5, 55, 1, -60, 8, 9, 3, 50, -50, 2]

# Valid readings
valid = [x for x in readings if -50 <= x <= 50]

# Convert Celsius to Fahrenheit
fahrenheit = [(x * 9/5) + 32 for x in valid]

# Alarm readings
alarms = [
    (i + 1, temp)
    for i, temp in enumerate(readings)
    if -50 <= temp <= 50 and (temp < 2 or temp > 8)
]

print("Valid:", valid)
print("Fahrenheit:", fahrenheit)
print("Alarms:", alarms)
```

```
Valid: [4, 5, 1, 8, 9, 3, 50, -50, 2]
Fahrenheit: [39.2, 41.0, 33.8, 46.4, 48.2, 37.4, 122.0, -58.0, 35.6]
Alarms: [(4, 1), (7, 9), (9, 50), (10, -50)]
```

PYTHON PROGRAMMING
C0_3 ASSINGMENT

5.USING MAP(), FILTER() AND LAMBDA

```
readings = [4, 5, 55, 1, -60, 8, 9, 3, 50, -50, 2]

# Filter valid readings
valid = list(filter(lambda x: -50 <= x <= 50, readings))

# Convert into Fahrenheit
fahrenheit = list(
    map(lambda x: x * 9/5 + 32, valid)
)

# Find alarm readings
alarms = list(
    filter(lambda x: x < 2 or x > 8, valid)
)

print("Valid:", valid)
print("Fahrenheit:", fahrenheit)
print("Alarm readings:", alarms)
```

```
Valid: [4, 5, 1, 8, 9, 3, 50, -50, 2]
Fahrenheit: [39.2, 41.0, 33.8, 46.4, 48.2, 37.4, 122.0, -58.0, 35.6]
Alarm readings: [1, 9, 50, -50]
```

6.GENERAL FORM OF LIST COMPREHENSION

main.py

```
numbers = [1, 2, 3, 4, 5, 6]

even = [x for x in numbers if x % 2 == 0]

print(even)
```

```
[2, 4, 6]
```

7.DICTIONARY AND SET COMPREHENSION

```
readings = [4, 5, 1, 8, 9, 3]

# Dictionary comprehension
data = {i+1: temp for i, temp in enumerate(readings)}

# Set comprehension
unique = {x for x in readings}

print(data)
print(unique)
```

```
{1: 4, 2: 5, 3: 1, 4: 8, 5: 9, 6: 3}
{1, 3, 4, 5, 8, 9}
```

PYTHON PROGRAMMING
C0_3 ASSINGMENT

8.DIFFERENCE BETWEEN LIST COMPREHENSION AND GENERATOR EXPRESSION

```
rnumbers = [1, 2, 3, 4, 5]

a = [x*x for x in numbers]
b = (x*x for x in numbers)

print(a)
print(list(b))
```

```
{1: 4, 2: 5, 3: 1, 4: 8, 5: 9, 6: 3}
{1, 3, 4, 5, 8, 9}
```

9.EXPERIMENTAL INVESTIGATION

```
 rimport random
 import time
 import sys

def loop_method(data):

    result = []

    for x in data:
        if -50 <= x <= 50:
            result.append(x)

    return result


def comprehension_method(data):

    return [x for x in data if -50 <= x <= 50]


def map_filter_method(data):
```

PYTHON PROGRAMMING
C0_3 ASSINGMENT

10.COMPLEXITY ANALYSIS

Method

Time

Extra memory

For loop

O(n)

O(n)

List comprehension

O(n)

O(n)

map/filter

O(n)

O(n) when converted to list

Generator

O(n) total

O(1)

Q6 – Nested Data: Railway Timetable (7 Marks)

1.COMPLETE PYTHON PROGRAM

PYTHON PROGRAMMING
C0_3 ASSINGMENT

```python
# Railway Timetable

timetable = {

    "12001": {
        "name": "Chennai Express",
        "stops": [
            ("Chennai", "08:00", "08:05"),
            ("Tambaram", "08:45", "08:50"),
            ("Villupuram", "10:00", "10:05")
        ],
        "days": {"Mon", "Wed", "Fri"}
    },

    "12002": {
        "name": "Pandian Express",
        "stops": [
            ("Chennai", "09:00", "09:10"),
            ("Tambaram", "09:40", "09:45"),
            ("Madurai", "14:00", "14:05")
        ],
        "days": {"Tue", "Thu", "Sat"}
```

```
Trains at Chennai on Monday:
12003 Vaigai Express 07:05
12001 Chennai Express 08:05
12004 Rockfort Express 10:05
```

2.INPUT

```
Station: Chennai
Day: Mon
```

3.OUTPUT

```
Trains at Chennai on Monday:

12003 Vaigai Express 07:05
12001 Chennai Express 08:05
12004 Rockfort Express 10:05
```

4. EXPRESSION TO ACCESS THE DEPARTURE TIME OF THE THIRD STATION

```python
timetable["12001"]["stops"][2][2]
```

5.WHY USE DIFFERENT STRUCTURES

Data structure

PYTHON PROGRAMMING
C0_3 ASSINGMENT

Purpose

Dictionary

Stores train details

List

Stores stations in order

Tuple

Stores station, arrival and departure

Set

Stores running days

6.CREATE A STATION BOX

```
station_index = {}
for number, train in timetable.items():
    for stop, arrival, departure in train["stops"]:
        station_index.setdefault(stop, []).append(number)

print("Station Index:")
for station, trains in station_index.items():
    print(station, ":", trains)
```

7.FIND TRAINS USING STAION INDEX

```python
def indexed_search(station, day):

    result = []

    for number in station_index.get(station, []):

        train = timetable[number]

        if day in train["days"]:

            for stop, arrival, departure in train["stops"]:

                if stop == station:

                    result.append(
                        (number, train["name"], departure)
                    )

    return sorted(result, key=lambda x: x[2])


print(indexed_search("Chennai", "Mon"))
```

8.COMPLEXITY ANALYSIS

Let:

- T = total number of trains.
- S = average number of stops per train.
- K = number of trains stopping at the requested station.

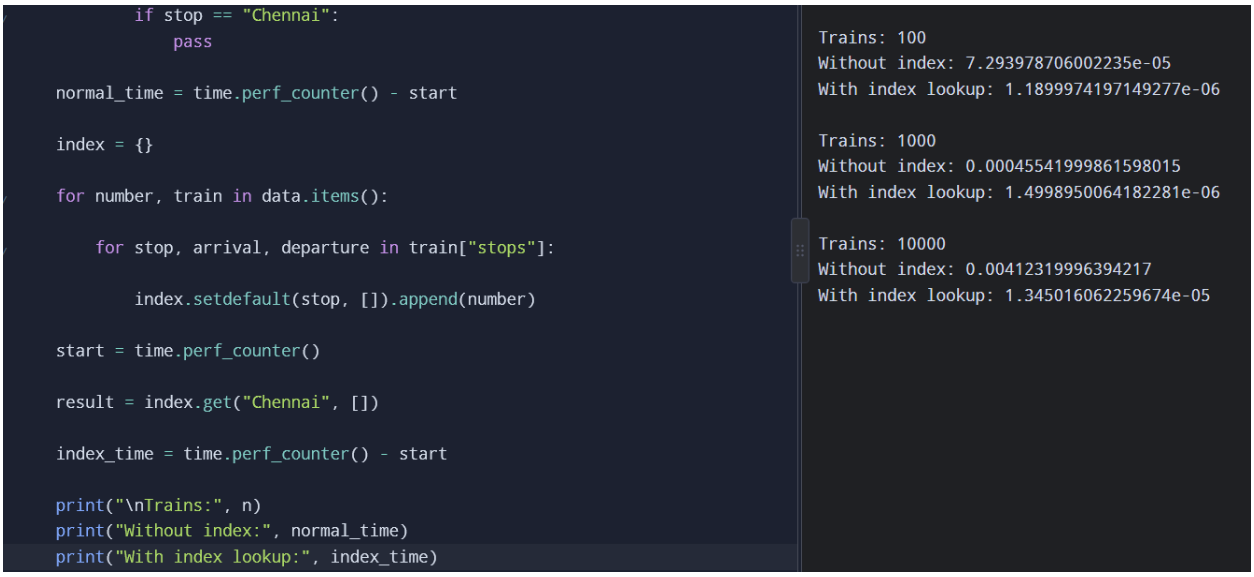
Method	Time complexity
Without station index	O(T × S)

PYTHON PROGRAMMING
C0_3 ASSINGMENT

With station index	O(K × S) in this implementation
Sorting results	O(K log K)
Building index	O(T × S)

The index requires additional O(T × S) space in the worst case.

9.EXPERIMENTAL INVESTIGATION



```
            if stop == "Chennai":
                pass

    normal_time = time.perf_counter() - start

    index = {}

    for number, train in data.items():

        for stop, arrival, departure in train["stops"]:

            index.setdefault(stop, []).append(number)

    start = time.perf_counter()

    result = index.get("Chennai", [])

    index_time = time.perf_counter() - start

    print("\nTrains:", n)
    print("Without index:", normal_time)
    print("With index lookup:", index_time)
```

```
Trains: 100
Without index: 7.293978706002235e-05
With index lookup: 1.1899974197149277e-06

Trains: 1000
Without index: 0.00045541999861598015
With index lookup: 1.4998950064182281e-06

Trains: 10000
Without index: 0.00412319996394217
With index lookup: 1.345016062259674e-05
```

Q7 – Design a Contact-Merging Program from First Principles (10 Marks)

1.STAGE 1-UNDERSTAND THE PROBLEM

Input:

- Contacts from two old phones.
- Contacts from a SIM card.

Output:

- One record for each person.
- All unique phone numbers.
- All unique email addresses.

PYTHON PROGRAMMING
C0_3 ASSINGMENT

Rules:

1. Normalize phone numbers before comparing.
2. Remove spaces, hyphens and unnecessary symbols.
3. Use a dictionary to store merged contacts.
4. Use sets to avoid duplicate phone numbers and emails.
5. Do not merge two different people just because their names are the same.

2.STAGE 2-DESIGN THE SOLUTION

Data structures:

Data structure	**Purpose**
Dictionary	Stores merged contacts
Set	Stores unique phone numbers
Set	Stores unique email addresses
List	Stores input contacts

Phone number normalization:

For example:

```
+91 98400 12345
```

3.COMPLETE THE PYTHON PROGRAM

PYTHON PROGRAMMING
C0_3 ASSINGMENT

```
    key = (
        sorted(data["phones"])[0]
        if data["phones"]
        else sorted(data["emails"])[0]
    )

    merged[key] = {
        "name": name,
        "phones": sorted(data["phones"]),
        "emails": sorted(data["emails"])
    }


# Display output

print("Merged Contacts:")

for key, value in merged.items():

    print("\nName:", value["name"])
    print("Phones:", value["phones"])
    print("Emails:", value["emails"])
```

```
Merged Contacts:

Name: RAVI KUMAR
Phones: ['919840012345', '919840012346']
Emails: ['ravi.kumar@gmail.com', 'ravi@gmail.com']

Name: Ananya
Phones: ['919876543210']
Emails: ['ananya@gmail.com']

Name: Ravi
Phones: ['919123456780']
Emails: ['another.ravi@gmail.com']
```

4.SAMPLE INPUT

```
Dr. Ravi
+91 98400 12345
ravi@gmail.com

Ravi Kumar
9840012345
9840012346
ravi@gmail.com

RAVI KUMAR
9840012346
ravi.kumar@gmail.com

Ananya
9876543210
ananya@gmail.com

Ravi
9123456780
another.ravi@gmail.com
```

5.EXPECTED OUTCOME

```
Merged Contacts:

Name: Ravi Kumar
Phones: ['919840012345', '919840012346']
Emails: ['ravi.kumar@gmail.com',
         'ravi@gmail.com']

Name: Ananya
Phones: ['919876543210']
Emails: ['ananya@gmail.com']

Name: Ravi
Phones: ['919123456780']
Emails: ['another.ravi@gmail.com']
```

6.STAGE 3-MEASURE EXCEUTION TIME

PYTHON PROGRAMMING
C0_3 ASSINGMENT

```python
import time

for n in [1000, 100000]:

    test_contacts = []

    for i in range(n):

        test_contacts.append({
            "name": "Person" + str(i),
            "phones": ["900000" + f"{i:04d}"],
            "emails": [f"user{i}@gmail.com"]
        })

    start = time.perf_counter()

    parent = list(range(n))

    owner = {}

    for i, contact in enumerate(test_contacts):
```